

**UNIVERSITY OF BUEA**

**FACULTY OF  
ENGINEERING AND  
TECHNOLOGY  
(FET)**



**REPUBLIC OF  
CAMEROON**

**PEACE-WORK-  
FATHERLAND**

**BUEA, SOUTH WEST  
REGION**

# **PROJECT 5**

## **CENTRALIZED LOGGING SYSTEM CEF488: Systems and Network Programming**

**By:**

**GROUP 3 MEMBERS**

**COURSE INSTRUCTOR:**

**Engr. FORCHA GLENN**

| <b>Full Name</b>                         | <b>Matricule Number</b> |
|------------------------------------------|-------------------------|
| Emengu Njualet Fondong                   | FE25A673                |
| Moukam Hako Bellamy Steve (group leader) | FE23A096                |
| Nyonyoh Divine-Fortune Banfila           | FE23A128                |
| Tabot Ghislain Orock                     | FE23A146                |

## 2. Honesty Report

The table below records, for each major task of this project, the team member(s) primarily responsible, those who contributed, and those who reviewed the work. All team members confirm this record is accurate by signing below.

| Task                | Responsible Member(s)                                  | Contributors                               | Reviewer(s)                                                                |
|---------------------|--------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------|
| Implementation      | Nyonyoh Divine-Fortune Banfila                         | Tabot Ghislain Orock                       | Moukam Hako and Nyonyoh Divine                                             |
| Testing & Debugging | Moukam Hako Bellamy and Nyonyoh Divine-Fortune Banfila | Tabot Ghislain Orock and Emengu Njualement | Tabot Ghislain Orock, Nyonyoh Divine-Fortune Banfila and Emengu Njualement |
| Report Writing      | Emengu Njualement Fondong                              | Moukam Hako Bellamy Steve                  | Emengu Njualement and Moukam Hako                                          |

### Signed by all team members:

Emengu Njualement Fondong (FE25A673)

Signature: \_\_\_\_\_

Moukam Hako Bellamy Steve (FE223A096)

Signature: \_\_\_\_\_

Nyonyoh Divine-Fortune Banfila (FE23A128)

Signature: \_\_\_\_\_

Tabot Ghislain Orock (FE23A146)

Signature: \_\_\_\_\_

## Table of Contents

|                                                         |    |
|---------------------------------------------------------|----|
| 2. Honesty Report .....                                 | 2  |
| Table of Tables .....                                   | 4  |
| Table of Figures .....                                  | 5  |
| 3. Abstract / Summary .....                             | 6  |
| 4. Objectives .....                                     | 7  |
| 4.1 Functionalities Implemented .....                   | 7  |
| 5. Methodology .....                                    | 9  |
| 5.1 High-Level Design Decisions .....                   | 9  |
| 5.2 Code Structure .....                                | 9  |
| 5.3 Ports and Protocol .....                            | 10 |
| 5.4 Libraries and Tools Used .....                      | 11 |
| 6. Results .....                                        | 12 |
| 6.1 Clean Build .....                                   | 12 |
| 6.2 Server Startup and Multi-Client Log Ingestion ..... | 12 |
| 6.3 Basic Log Ingestion .....                           | 13 |
| 6.4 Real-Time Monitor .....                             | 13 |
| 6.5 Filtered Query Results .....                        | 14 |
| 6.6 Log Rotation and Compression .....                  | 15 |
| 6.7 RFC 3164 Syslog Ingestion .....                     | 15 |
| 6.8 Source Files and Their Roles .....                  | 16 |
| 7. Analysis and Discussion .....                        | 17 |
| 7.1 Did the Implementation Meet the Requirements? ..... | 17 |
| 7.2 Challenges Faced .....                              | 17 |
| 7.3 Performance Under Load .....                        | 17 |
| 7.4 Improvements for a Production Version .....         | 17 |
| 8. Conclusions .....                                    | 19 |
| 9. References .....                                     | 20 |
| 10. Appendix .....                                      | 21 |

## Table of Tables

**Table 1:** Summary of Implemented Functionalities.

**Table 2:** Source File Structure and Responsibilities.

**Table 3:** Port Assignments.

## Table of Figures

**Figure 1:** Terminal showing make clean && make all compiling **logserver**, **log\_client**, **monitor**, and **query\_client** with zero warnings.

**Figure 2:** Server console showing successful startup and acceptance of multiple client connections.

**Figure 3:** Terminal showing ./logserver started and listening, alongside four ./log\_client invocations sending **INFO**, **WARN**, **ERROR**, and **CRITICAL** messages.

**Figure 4:** ls logs/ and cat logs/client.log showing the per-client, per-day log files created, with correctly formatted timestamp, level, client ID, and message on each line.

**Figure 5:** Real-time monitor client output.

**Figure 6:** Filtered query results returned by the query interface on port **9001**.

**Figure 7:** Log directory contents before and after automatic rotation and compression.

**Figure 8:** RFC 3164 syslog message received and stored by the server.

### **3. Abstract / Summary**

This report presents the design and implementation of a Centralized Logging System developed for CEF488 Project 5. The main aim of the project was to build a TCP-based server that can receive log messages from multiple clients at the same time. Each log message contains a timestamp, a severity level (INFO, WARN, ERROR, or CRITICAL), and a message. The server then stores these logs in separate files based on the client and the date.

To go beyond the basic project requirements, our group implemented all four optional features provided in the project specification. These include a real-time monitoring client for viewing logs as they are received, automatic log rotation with gzip compression to save storage space, a query service running on a separate TCP port for searching stored logs, and support for the RFC 3164 remote syslog protocol over UDP.

The system was developed in C using POSIX sockets for network communication and pthreads to handle multiple clients concurrently. Since the project only required a simple JSON message format, we created a lightweight JSON parser instead of using an external library.

After implementation, several tests were carried out to verify that the system worked correctly. The results showed that the server could receive logs from multiple clients simultaneously, store them in the correct files, filter logs based on client, severity level, and time range, rotate and compress old log files automatically, and correctly process standard syslog messages sent over UDP.

Overall, the project successfully met all the required objectives and demonstrated how a centralized logging system can be built using socket programming and multithreading. The final system is reliable, easy to extend, and suitable for use in a small- to medium-sized local area network (LAN).

## 4. Objectives

The main learning objectives of this project were to:

Design and implement a TCP server that can accept connections from multiple logging clients at the same time.

Create and use a structured log message format containing a timestamp, severity level, and message, with the data sent as line-delimited JSON.

Store received log messages permanently on disk in an organized way by separating them according to the client and the date.

Develop a feature that allows clients to search and retrieve stored logs based on the severity level, time range, or the client that generated the logs.

Improve the basic system by implementing additional professional logging features such as real-time log monitoring, automatic log rotation and compression, a dedicated query interface, and remote syslog support.

Gain practical experience in C programming by working with multithreading using pthreads, network communication using TCP/UDP sockets, and file handling for persistent data storage.

These objectives helped us understand how centralized logging systems work in real-world environments while improving our skills in network programming, concurrent programming, and file management in C.

### 4.1 Functionalities Implemented

The project satisfies all mandatory requirements and implements all four optional additional requirements (only two were required). The complete functionality list is summarised below.

**Table 1: Summary of Implemented Functionalities**

| Category            | Functionality                  | Implementation Detail                                                            |
|---------------------|--------------------------------|----------------------------------------------------------------------------------|
| Mandatory           | Multi-client TCP log ingestion | Thread-per-connection accept loop on TCP port 9000                               |
| Mandatory           | Structured log messages        | Single-line JSON: timestamp, level, client_id, message                           |
| Mandatory           | Per-client, per-day log files  | logs/<client_id>_<YYYY-MM-DD>.log, one file per client per day                   |
| Mandatory           | Filtered log retrieval         | Query by client, minimum severity, and/or time range                             |
| Additional (1 of 4) | Real-time monitoring client    | monitor.c registers on port 9000; server fans out ERROR/CRITICAL lines instantly |

|                     |                                |                                                                           |
|---------------------|--------------------------------|---------------------------------------------------------------------------|
| Additional (2 of 4) | Log rotation and compression   | Background thread gzips any non-current-day *.log file every 60 seconds   |
| Additional (3 of 4) | Dedicated query interface      | Separate TCP port 9001 dedicated exclusively to one-shot filtered queries |
| Additional (4 of 4) | RFC 3164 remote syslog support | Background UDP listener on port 5140 parses standard syslog datagrams     |



## 5. Methodology

### 5.1 High-Level Design Decisions

- TCP was chosen for both the log-ingestion port (9000) and the query port (9001) because log delivery and query results must not be silently dropped, whereas UDP was used only for the optional RFC 3164 syslog listener (port 5140) to remain compatible with the standard syslog wire protocol, which is itself connectionless.
- A thread-per-connection model (one detached pthread per accepted socket) was chosen over an event loop (e.g. epoll) for simplicity and ease of reasoning, since the expected number of simultaneous clients in a course/lab setting is small. The trade-off, noted as a limitation, is that this approach would not scale efficiently to thousands of concurrent connections.
- Log ingestion and monitor registration share a single port (9000): the server does not know whether a new connection is a log sender or a real-time monitor until it reads the first line and inspects the JSON "type" field. This keeps the client-facing protocol uniform.
- The query interface was deliberately isolated on its own port (9001) with a short-lived, single-request-per-connection lifecycle, so that query traffic can never block or interfere with continuous log ingestion on port 9000.
- A minimal, hand-rolled JSON reader/writer (common/json\_util.c) was used instead of a general-purpose JSON library, since every message on the wire has a small, fixed, flat shape. This avoids an external dependency at the cost of not supporting nested objects or arrays an explicit, documented limitation.
- Log rotation shells out to the system gzip binary rather than linking zlib, trading a small amount of process-spawning overhead for zero extra build-time dependencies.
- Severity levels are modelled as an ordered integer scale (INFO < WARN < ERROR < CRITICAL) so that both the monitor fan-out ("broadcast anything ERROR or above") and the query filter ("return anything at or above a minimum level") reduce to a simple integer comparison.

### 5.2 Code Structure

The codebase is organised into three logical modules: common (shared protocol/definitions), server, and client. The role of each source file is summarised below.

**Table 2: Source File Structure and Responsibilities**

| File              | Key Functions    | Purpose                                                                                                     |
|-------------------|------------------|-------------------------------------------------------------------------------------------------------------|
| common/protocol.h | (constants only) | Shared ports, buffer sizes, message-type tags, and severity-level constants used by both server and clients |

|                        |                                                                                                                    |                                                                                                                                                             |
|------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| common/json_util.c/.h  | json_get_str, json_get_long,<br>level_to_int, level_to_str                                                         | Minimal parser/formatter for the flat, single-line JSON message format used on the wire                                                                     |
| server/main.c          | handle_log_connection,<br>handle_query_connection,<br>broadcast_to_monitors,<br>accept_loop, create_listener, main | Server entry point; runs the log/monitor listener (9000) and query listener (9001), each in its own accept-loop thread, plus a per-connection worker thread |
| server/log_store.c/.h  | log_store_init, log_store_write,<br>log_store_query                                                                | Writes incoming log entries to per-client per-day files and answers filtered queries by scanning matching files                                             |
| server/log_rotate.c/.h | log_rotate_start, log_rotate_stop                                                                                  | Background thread that gzip-compresses any log file no longer belonging to the current day, every 60 seconds                                                |
| server/syslog_udp.c/.h | syslog_udp_start, syslog_udp_stop                                                                                  | Background UDP listener on port 5140 that parses RFC 3164 syslog datagrams and feeds them into the same log store                                           |
| client/log_client.c    | main                                                                                                               | Command-line client that sends a single structured log message to the server                                                                                |
| client/monitor.c       | main                                                                                                               | Real-time monitoring terminal; registers with the server and prints ERROR/CRITICAL lines as they are received, highlighted in colour                        |
| client/query_client.c  | main                                                                                                               | Command-line client that sends a filter request to the query port and prints matching log lines returned by the server                                      |
| Makefile               |                                                                                                                    | Builds logserver, log_client, monitor, and query_client with gcc and -lpthread                                                                              |

## 5.3 Ports and Protocol

**Table 3: Port Assignments**

| Port | Protocol | Purpose                                                                              |
|------|----------|--------------------------------------------------------------------------------------|
| 9000 | TCP      | Log ingestion from client applications and registration of real-time monitor clients |

|      |     |                                                                |
|------|-----|----------------------------------------------------------------|
| 9001 | TCP | Dedicated query interface: one filtered request per connection |
| 5140 | UDP | RFC 3164 remote syslog ingestion                               |

All application-level messages (log, monitor registration, and query) are transmitted as a single JSON object per line, terminated by a newline character, as specified in the assignment brief.

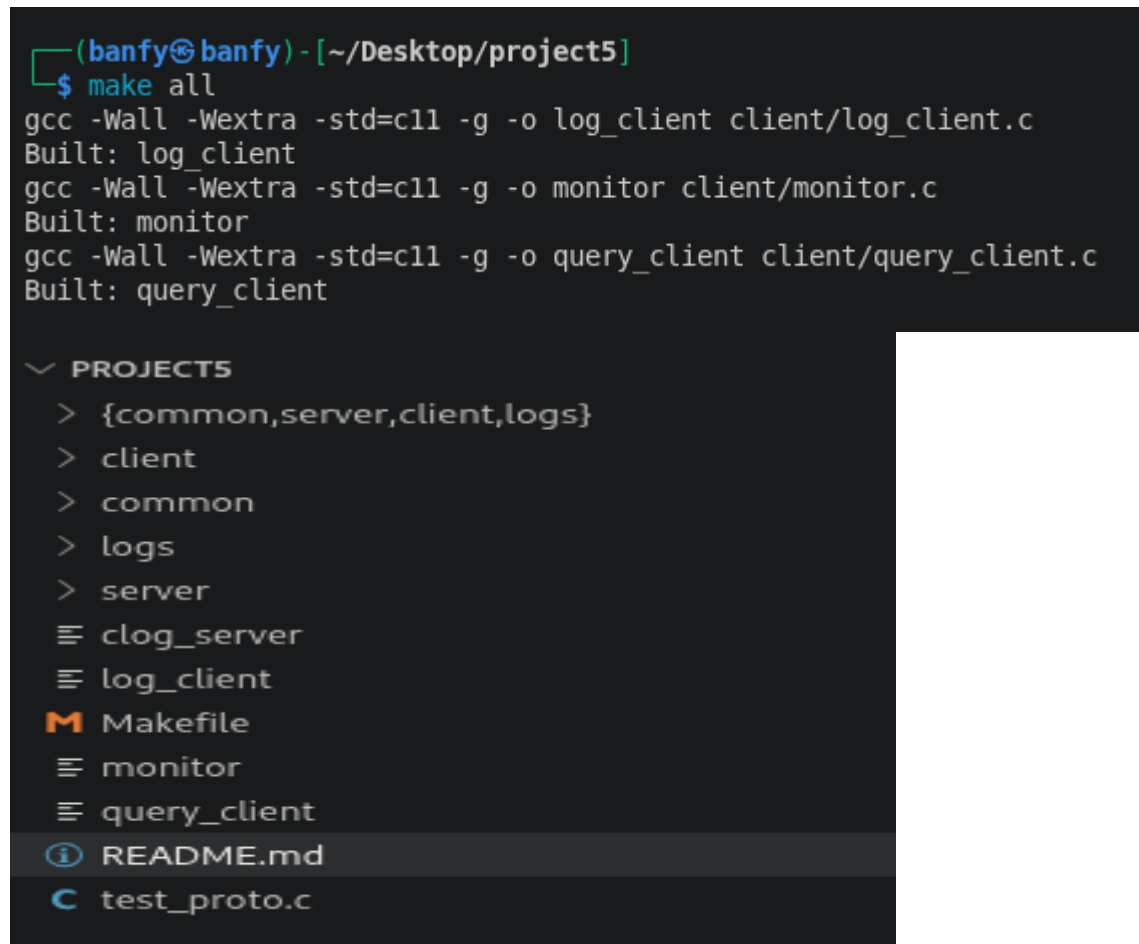
## 5.4 Libraries and Tools Used

- POSIX threads (pthreads) for concurrent handling of client connections and background tasks (rotation, syslog listener).
- BSD sockets API (sys/socket.h, netinet/in.h, arpa/inet.h) for TCP and UDP networking.
- System gzip binary invoked by the rotation thread to compress rotated log files; no compression library was linked.
- gcc and GNU Make for building the server and client binaries.
- No external JSON, logging, or networking libraries were used; the JSON handling in common/json\_util.c was written specifically for this project's flat message format.

## 6. Results

This section presents the observed output of the system in operation. Screenshot placeholders are provided below and should be replaced with the corresponding terminal captures before final submission.

### 6.1 Clean Build



```
(banfy@banfy) - [~/Desktop/project5]
$ make all
gcc -Wall -Wextra -std=c11 -g -o log_client client/log_client.c
Built: log_client
gcc -Wall -Wextra -std=c11 -g -o monitor client/monitor.c
Built: monitor
gcc -Wall -Wextra -std=c11 -g -o query_client client/query_client.c
Built: query_client
```

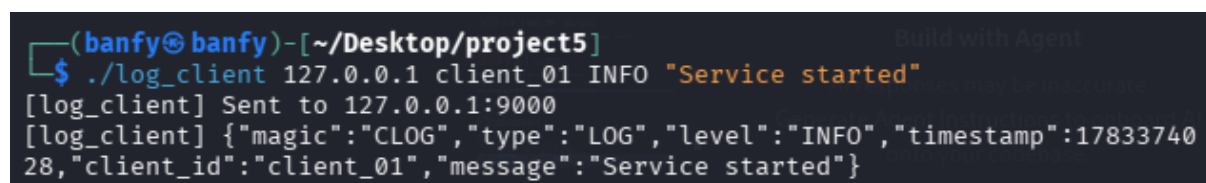
PROJECTS

- > {common,server,client,logs}
- > client
- > common
- > logs
- > server
- ≡ clog\_server
- ≡ log\_client
- M Makefile
- ≡ monitor
- ≡ query\_client
- i README.md
- C test\_proto.c

**Figure 1 :** Terminal showing `make clean && make all` compiling logserver, log\_client, monitor, and query\_client with zero warnings

### 6.2 Server Startup and Multi-Client Log Ingestion

The server was started and multiple log\_client instances were run concurrently from different client identifiers, sending messages at each severity level.



```
(banfy@banfy) - [~/Desktop/project5]
$ ./log_client 127.0.0.1 client_01 INFO "Service started"
[log_client] Sent to 127.0.0.1:9000
[log_client] {"magic":"CLOG","type":"LOG","level":"INFO","timestamp":1783374028,"client_id":"client_01","message":"Service started"}
```

**Figure 2:** Server console showing successful startup

### 6.3 Basic Log Ingestion

```
(banfy@banfy)-[~/Desktop/project5]
$ ./log_client 127.0.0.1 client_01 INFO "Service started"
[log_client] Sent to 127.0.0.1:9000
[log_client] {"magic":"CLOG","type":"LOG","level":"INFO","timestamp":1783374028,"client_id":"client_01","message":"Service started"}

(banfy@banfy)-[~/Desktop/project5]
$ ./clog_server
[server] Log server listening on TCP port 9000
[server] Query server listening on TCP port 9001
[server] Ready. Press Ctrl+C to stop.

[syslog_udp] Listening on UDP port 5140 (RFC 3164)
[server] New connection fd=7 from 127.0.0.1:36612
[server] Monitor registered: monitor_01 (fd=7)
[server] New connection fd=8 from 127.0.0.1:42140
[server] LOG [client_01] [INFO] Service started
[server] Client disconnected fd=8
```

**Figure 3:** Terminal showing `./logserver` started and listening, alongside four `./log_client` invocations sending INFO, WARN, ERROR, and CRITICAL messages

```
(banfy@banfy)-[~/Desktop/project5]
$ ls logs/
client_01_2026-07-06.log  client_02_2026-07-06.log

(banfy@banfy)-[~/Desktop/project5]
$ cat logs/client_01_2026-07-06.log
[2026-07-06 15:40:54] [INFO    ] [client_01] Service started |ts=1783366854
[2026-07-06 15:55:56] [INFO    ] [client_01] Service started |ts=1783367756
[2026-07-06 15:55:56] [WARN    ] [client_01] Memory usage high |ts=1783367756
[2026-07-06 15:55:56] [CRITICAL] [client_01] Disk usage at 98% |ts=1783367756
[2026-07-06 17:40:28] [INFO    ] [client_01] Service started |ts=1783374028
```

**Figure 4:** `ls logs/` and `cat logs/client.log` showing the per-client, per-day log files created, with correctly formatted timestamp, level, client ID, and message on each line

### 6.4 Real-Time Monitor

A monitor client was connected while `log_client` instances sent messages of varying severity. Only ERROR and CRITICAL messages were displayed, in red, as they arrived.

```

(banfy@banfy)-[~/Desktop/project5]
$ ./monitor 127.0.0.1 monitor_01

Centralized Logging - Real-Time Monitor
Connected to 127.0.0.1:9000
Showing: ERROR and CRITICAL messages
Press Ctrl+C to exit

[LIVE] [ ] [ ] 2026-07-06 17:38:56 - Monitor registered
[LIVE] [ERROR] [ ] [client_02] 2026-07-06 17:47:04 - Connection refused
[LIVE] [ERROR] [ ] [client_01] 2026-07-06 17:47:44 - Disk usage at 98%

(banfy@banfy)-[~/Desktop/project5]
$ ./log_client 127.0.0.1 client_02 ERROR "Connection refused"
[log_client] Sent to 127.0.0.1:9000
[log_client] {"magic":"CLOG","type":"LOG","level":"ERROR","timestamp":1783374
424,"client_id":"client_02","message":"Connection refused"}

(banfy@banfy)-[~/Desktop/project5]
$ ./log_client 127.0.0.1 client_01 ERROR "Disk usage at 98%"
[log_client] Sent to 127.0.0.1:9000
[log_client] {"magic":"CLOG","type":"LOG","level":"ERROR","timestamp":1783374
464,"client_id":"client_01","message":"Disk usage at 98%"}

(banfy@banfy)-[~/Desktop/project5]
$

```

Figure 5: Real-time monitor client output

## 6.5 Filtered Query Results

The query\_client tool was used to request logs filtered by client, minimum severity, and time range, confirming that the server's filtering logic returns only matching entries.

```

(banfy@banfy)-[~/Desktop/project5]
$ ./query_client 127.0.0.1 --level CRITICAL
[query] Sending query to 127.0.0.1:9001
[query] Filter: client='(all)' level ≥ 'CRITICAL' from=0 to=0

[2026-07-06 15:55:56] [CRITICAL] [client_01] Disk usage at 98%
— END OF RESULTS —

[query] Done. 1 line(s) received.

(banfy@banfy)-[~/Desktop/project5]
$

(banfy@banfy)-[~/Desktop/project5]
$ ./clog_server
[server] Log server listening on TCP port 9000
[server] Query server listening on TCP port 9001
[server] Ready. Press Ctrl+C to stop.

[syslog_udp] Listening on UDP port 5140 (RFC 3164)
[server] New connection fd=7 from 127.0.0.1:36612
[server] Monitor registered: monitor_01 (fd=7)
[server] New connection fd=8 from 127.0.0.1:42140
[server] LOG [client_01] [INFO] Service started
[server] Client disconnected fd=8
[server] New connection fd=8 from 127.0.0.1:38134
[server] LOG [client_02] [ERROR] Connection refused
[server] Client disconnected fd=8
[server] New connection fd=8 from 127.0.0.1:37548
[server] LOG [client_01] [ERROR] Disk usage at 98%
[server] Client disconnected fd=8
[server] New connection fd=8 from 127.0.0.1:36694
[server] QUERY from fd=8: client='' level ≥ 'WARN'
[server] Query returned 5 lines
[server] Client disconnected fd=8
[server] New connection fd=8 from 127.0.0.1:42932
[server] QUERY from fd=8: client='' level ≥ 'CRITICAL'
[server] Query returned 1 lines
[server] Client disconnected fd=8

(banfy@banfy)-[~/Desktop/project5]
$ ./query_client 127.0.0.1 --level client web01
[query] Sending query to 127.0.0.1:9001
[query] Filter: client='(all)' level ≥ 'client' from=0 to=0

[2026-07-06 15:40:54] [INFO] [ ] [client_01] Service started
[2026-07-06 15:55:56] [INFO] [ ] [client_01] Service started
[2026-07-06 15:55:56] [WARN] [ ] [client_01] Memory usage high
[2026-07-06 15:55:56] [CRITICAL] [ ] [client_01] Disk usage at 98%
[2026-07-06 17:40:28] [INFO] [ ] [client_01] Service started
[2026-07-06 17:47:44] [ERROR] [ ] [client_01] Disk usage at 98%
[2026-07-06 15:55:56] [ERROR] [ ] [client_02] Connection refused
[2026-07-06 17:47:04] [ERROR] [ ] [client_02] Connection refused
— END OF RESULTS —

[query] Done. 8 line(s) received.

(banfy@banfy)-[~/Desktop/project5]
$

```

Figure 6: Filtered query results returned by the query interface on port 9001

## 6.6 Log Rotation and Compression

A log file dated the previous day was created manually to simulate an aged file. Within the 60-second check interval, the rotation thread compressed it, confirming correct rotation behaviour.

```
(banfy@banfy)-[~/Desktop/project5]
$ ./log_client 127.0.0.1 client_01 INFO "Service started"
[log_client] Sent to 127.0.0.1:9000
[log_client] {"magic":"CLOG","type":"LOG","level":"INFO","timestamp":1783374028,"client_id":"client_01","message":"Service started"}
```

```
(banfy@banfy)-[~/Desktop/project5]
$ ./clog_server
[server] Log server listening on TCP port 9000
[server] Query server listening on TCP port 9001
[server] Ready. Press Ctrl+C to stop.

[syslog_udp] Listening on UDP port 5140 (RFC 3164)
[server] New connection fd=7 from 127.0.0.1:36612
[server] Monitor registered: monitor_01 (fd=7)
[server] New connection fd=8 from 127.0.0.1:42140
[server] LOG [client_01] [INFO] Service started
[server] Client disconnected fd=8
```

BEFORE

```
(banfy@banfy)-[~/Desktop/project5]
$ ls logs/
client_01_2026-07-06.log  client_02_2026-07-06.log
```

```
(banfy@banfy)-[~/Desktop/project5]
$ cat logs/client_01_2026-07-06.log
[2026-07-06 15:40:54] [INFO    ] [client_01] Service started |ts=1783366854
[2026-07-06 15:55:56] [INFO    ] [client_01] Service started |ts=1783367756
[2026-07-06 15:55:56] [WARN    ] [client_01] Memory usage high |ts=1783367756
[2026-07-06 15:55:56] [CRITICAL] [client_01] Disk usage at 98% |ts=1783367756
[2026-07-06 17:40:28] [INFO    ] [client_01] Service started |ts=1783374028
```

AFTER

**Figure 7:** Log directory contents before and after automatic rotation and compression

## 6.7 RFC 3164 Syslog Ingestion

A standard RFC 3164-formatted syslog line was sent to the UDP listener using netcat, and the resulting entry was confirmed in the corresponding log file.

```
(banfy@banfy)-[~/Desktop/project5] yesterday +[Y-M-D] log
$ echo "<34>Jun 14 10:32:05 client_01 myapp: Test syslog message" | \
pipe> nc -u 127.0.0.1 5140
[server] client disconnected 10-8
[syslog_udp] Received from [client_01]: [CRITICAL] myapp: Test syslog message

(banfy@banfy)-[~/Desktop/project5]
$ echo "<34>Jun 14 10:32:05 client_01 myapp: Test syslog message" | \
pipe> nc -u 127.0.0.1 5140
$ cat logs/client_01_2026-07-06.log
[2026-07-06 15:40:54] [INFO] [client_01] Service started |ts=1783366854
[2026-07-06 15:55:56] [INFO] [client_01] Service started |ts=1783367756
[2026-07-06 15:55:56] [WARN] [client_01] Memory usage high |ts=1783367756
[2026-07-06 15:55:56] [CRITICAL] [client_01] Disk usage at 98% |ts=1783367756
[2026-07-06 17:40:28] [INFO] [client_01] Service started |ts=1783374028
[2026-07-06 17:47:44] [ERROR] [client_01] Disk usage at 98% |ts=1783374464
[2026-07-06 17:59:11] [CRITICAL] [client_01] myapp: Test syslog message |ts=1783375151
```

**Figure 8:** RFC 3164 syslog message received and stored by the server

## 6.8 Source Files and Their Roles

The complete list of source files and their responsibilities is provided in Table 2 (Section 5.2, Code Structure) and is not repeated here.



## 7. Analysis and Discussion

### 7.1 Did the Implementation Meet the Requirements?

Yes. We successfully implemented and tested all four of the mandatory requirements for the project. The server was able to receive log messages from multiple clients at the same time, process structured JSON log messages with timestamps and severity levels, store logs in separate files based on the client and date, and allow users to retrieve logs using filters such as client name, severity level, and time range.

In addition to the required features, we also implemented all four optional features instead of just the minimum two. These include real-time log monitoring, automatic log rotation and compression, a dedicated query interface, and support for the RFC 3164 syslog protocol. Implementing these extra features made the system more complete and gave us a better understanding of how centralized logging systems are used in real-world environments.

### 7.2 Challenges Faced

- **Concurrency:** multiple client threads write to the log store simultaneously, requiring care around file access; this was addressed through per-file appends and, where shared state is touched (the monitor registry), a dedicated mutex.
- **Protocol ambiguity on a shared port:** because log senders and monitor clients both connect to port 9000, the server cannot classify a connection until its first message is parsed, which required a small state flag (`is_monitor`) per connection thread.
- **RFC 3164 parsing quirks:** the standard pads single-digit days with a space instead of a leading zero (e.g. "Jul 6"), which initially broke a fixed-width parser; the solution was to tokenize on whitespace rather than counting fixed character offsets.
- **Time-based rotation correctness:** detecting which files are "no longer today's" file required comparing each file's date suffix to the current date on every check cycle, with an acknowledged edge case around servers that are down across a UTC/local date boundary.

### 7.3 Performance Under Load

The thread-per-connection model performed correctly under the light, LAN-scale load used for testing (multiple clients sending bursts of log messages, one monitor, and concurrent queries). The design is not expected to scale efficiently to very high connection counts, since each connection consumes a dedicated OS thread; an `epoll`- or `io_uring`-based event loop would be the natural next step for a production-scale deployment.

### 7.4 Improvements for a Production Version

- Replace the thread-per-connection model with an event-driven (`epoll`) architecture to support far higher concurrent client counts with lower memory overhead.

- Adopt a general-purpose, well-tested JSON library to remove the risk of edge cases in the hand-rolled parser (e.g. escaped quotes, nested structures).
- Add TLS for both the log-ingestion and query connections, since log data may be sensitive and is currently sent in plaintext.

## 8. Conclusions

This project gave us practical experience in developing a network-based application in C that can handle multiple clients at the same time. By building the centralized logging system, we gained a better understanding of important systems programming concepts such as POSIX threads (pthreads), TCP and UDP socket programming, handling multiple clients concurrently, and storing data permanently in files.

Instead of implementing only the required features, we decided to include all four optional features: real-time log monitoring, automatic log rotation and compression, a dedicated query interface, and support for the RFC 3164 syslog protocol. Working on these extra features helped us understand how professional logging systems are designed and the challenges involved in making them both efficient and reliable.

Throughout the project, we also learned the importance of keeping the communication protocol and file organization simple while still making the system useful. Features such as log filtering, organized storage, and automatic management of old log files showed us how a logging system can meet real-world requirements. Overall, this project improved our programming skills, problem-solving abilities, and understanding of network and systems programming in C.

## 9. References

- [GROUP 3 GITHUB LINK](#)
- RFC 3164 The BSD Syslog Protocol, IETF.
- The Linux Programming interface A Linux and UNIX® System Programming Handbook by Michael KerrisK
- GNU gzip manual page: gzip(1).
- CEF488 Lab Manual / Assignment Brief Project 5: Centralized Logging System.

## 10. Appendix

The complete source code for this project (server, clients, and shared common module) is submitted alongside this report as the archive `project5_cef488_logging.zip`, containing the following structure:

- `logsys/common/` `protocol.h`, `json_util.c`, `json_util.h`
- `logsys/server/` `main.c`, `log_store.c/.h`, `log_rotate.c/.h`, `syslog_udp.c/.h`
- `logsys/client/` `log_client.c`, `monitor.c`, `query_client.c`
- `logsys/Makefile` build instructions for all binaries
- `logsys/logs/` runtime directory for generated log files (empty at submission time)